TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico de Veracruz

Ingeniería en Sistemas Computacionales

Materia: Base de Datos NoSQL

Documento de Diseño Técnico de la Base de Datos, Memoria Técnica de Diseño y Migración a NoSQL (MongoDB Atlas)

Proyecto: SIGEM — Sistema Integral de Gestión de Equipos y Materiales

Integrantes:
Rafael Rayón Hernández
Francisco Alejandro Parra Huerta

Repositorio: sigem-tecnm-veracruz (rama migracion-mongodb)

Veracruz, Ver., octubre de 2026

Índice

1. Contenido 4
1.1 Introducción 4
1.1.1 Limitaciones del modelo relacional previo 4
1.1.2 Justificación de la migración a MongoDB Atlas 4
1.2 Estrategia de modelado: referenciación vs. embebimiento 4
1.3 Validación de esquema e índices (DBA) 5
1.3.1 JSON Schema Validation ($jsonSchema) 5
1.3.2 Estrategia de indexación 5
1.4 Pipelines de agregación implementados 5
1.4.1 Pipeline 1: Cruce de solicitudes con usuario y métricas ($lookup + $unwind) 6
1.4.2 Pipeline 2: Análisis de activos por estado y modalidad ($group + $match) 6
1.5 Seguridad y hardening en MongoDB Atlas 7
1.5.1 Control de acceso basado en roles (RBAC) 7
1.5.2 Cifrado en tránsito y en reposo 7
1.5.3 Hardening de red (IP Access List) 7
1.6 Integración con GitHub Student Developer Pack 7
1.6.1 Beneficio 1: MongoDB Atlas (créditos estudiantiles) 7
1.6.2 Beneficio 2: Namecheap (dominio .me y gestión DNS) 8
2. Diagrama de Documentos 9
2.1 Documento: inventario 9
2.2 Documento: solicitud (patrón embebido) 10
3. Diagrama de Modelo de Datos (Collections) 11
3.1 Diccionario de colecciones 12
users 12
material 12
inventario 12
solicitud 13
receptor 13
area 13
departamento 13
4. Diagramas de Arquitectura de Mongo 14
4.1 Flujo de comunicación 15
5. Vistas de Figma 16
5.1 Vista Administrador 16

.....16
5.2 Vista Servicio Social.....17
5.3 Vistas adicionales.....18
6. Conclusión.....20

1. Contenido

1.1 Introducción

El proyecto SIGEM fue concebido para solventar la problemática de dispersión, pérdida de información y lentitud en el préstamo y control de inventario de equipos y materiales del Campus Veracruz. Originalmente, el sistema fue modelado sobre una base de datos relacional tradicional (MySQL 8), sustentada en 15 tablas vinculadas mediante llaves foráneas e índices B-Tree.

1.1.1 Limitaciones del modelo relacional previo

1. **Rigidez de esquema:** Los activos del instituto presentan atributos dinámicos dispares (ej. una laptop cuenta con procesador, memoria RAM y dirección MAC, mientras que un consumible o herramienta carece de número de serie y se maneja por lote/volumen). En MySQL, esto obligaba al uso de tablas intermedias con campos nulos o esquemas EAV (Entity-Attribute-Value) de alto costo computacional.
2. **Degradación por operaciones JOIN complejas:** Las consultas para generar reportes ejecutivos de solicitudes y préstamos requerían cruzar hasta 7 tablas simultáneas (solicitud, detalle_solicitud, users, inventario, material, marca_material, departamento), penalizando la latencia en horas pico de servicio.
3. **Mantenimiento y costos de infraestructura:** La administración de instancias MySQL dedicadas implicaba gestión manual de parches, backups y balanceadores, con escasa elasticidad ante incrementos repentinos de tráfico.

1.1.2 Justificación de la migración a MongoDB Atlas

- **Modelo documental orientado al dominio:** Los documentos BSON permiten modelar la entidad solicitud con sus items solicitados como subdocumentos embebidos directos, eliminando la sobrecarga de consultas en tablas de unión.
- **Escalabilidad horizontal y alta disponibilidad:** MongoDB Atlas implementa clústeres basados en Replica Sets de tres nodos (un primario y dos secundarios distribuidos en distintas zonas de disponibilidad de AWS), garantizando conmutación por error (failover) automática en menos de 5 segundos.
- **ODM oficial y ecosistema Laravel:** La adopción del paquete mongodb/laravel-mongodb permite desacoplar los modelos de Laravel preservando la expresividad del patrón Active Record / Eloquent, a la vez que habilita el marco de agregaciones nativo ($lookup, $group, $facet).

1.2 Estrategia de modelado: referenciación vs. embebimiento

En entornos NoSQL, el principio rector es: "Lo que se consulta junto, se almacena junto". En SIGEM se implementó una estrategia híbrida:

1. **Referenciación mediante `ObjectId` (normalización parcial):** utilizada para entidades independientes de alta mutabilidad y catálogos globales reutilizados en múltiples módulos (users, departamento, area, material, proveedor, receptor). Evita la duplicación incontrolada de datos y asegura que un cambio de nombre en un proveedor o material se refleje inmediatamente.

2. **Embebimiento documental (desnormalización estratégica):** implementado en solicitud.detalles, donde cada solicitud almacena dentro de un arreglo BSON los items requeridos (id_material, cantidad, observaciones); al abrir un préstamo, todo el detalle se recupera en una sola lectura. También en historial_estados: los cambios de custodia y estado pueden integrarse secuencialmente como subdocumentos dentro de inventario.

1.3 Validación de esquema e índices (DBA)

Para garantizar la integridad semántica de los datos en ausencia de un motor relacional rígido, se aplicaron directivas nativas de MongoDB.

1.3.1 JSON Schema Validation ($jsonSchema)

Implementado mediante el script maestro database/mongo/init_database.js. Cada documento insertado es validado en el servidor Atlas:

- **Tipos BSON restrictivos:** objectId, string, date, bool, int.
- **Restricciones de negocio:** enums obligatorios para tipo_usuario (Administrador, Servicio Social), estado de inventario (Disponible, Asignado, En Mantenimiento, Dañado, Baja), y valores mínimos minimum: 0 para existencias en inventario.

1.3.2 Estrategia de indexación

Colección	Campo(s) indexado(s)	Tipo de índice	Justificación técnica
users	email	Unique Index	Previene registros duplicados en autenticación.
users	tipo_usuario, activo	Compound Index	Acelera filtrado de usuarios activos por panel.
inventario	num_serie	Unique Index	Integridad del número de placa/activo.
inventario	id_producto, estado	Compound Index	Optimiza conteo de stock disponible por material.
inventario	ubicacion_fisica	Single Field	Búsqueda rápida durante auditorías presenciales.
material	stock_actual, stock_minimo	Compound Index	Optimiza la detección de alertas de reabastecimiento.
solicitud	fecha_solicitud	Descending (-1)	Acelera la paginación de la bitácora más reciente.
solicitud	estado, fecha_solicitud	Compound Index	Carga instantánea de solicitudes pendientes en el panel.

1.4 Pipelines de agregación implementados

Para cubrir la necesidad de reportes analíticos complejos sin transferir grandes volúmenes de datos hacia PHP, se diseñaron pipelines nativos en el motor de MongoDB.

1.4.1 Pipeline 1: Cruce de solicitudes con usuario y métricas ($lookup + $unwind)

Une la colección solicitud con los datos del usuario solicitante en users, proyectando el resumen del trámite:

```
targetDb.solicitud.aggregate([
  { $match: { estado: "Pendiente" } },
  {
    $lookup: {
      from: "users",
      localField: "id_usuario",
      foreignField: "_id",
      as: "usuario_solicitante"
    }
  },
  { $unwind: "$usuario_solicitante" },
  {
    $project: {
      _id: 1,
      tipo_movimiento: 1,
      estado: 1,
      fecha_solicitud: 1,
      total_items: { $size: { $ifNull: ["$detalles", []] } },
      solicitante_nombre: "$usuario_solicitante.name",
      solicitante_email: "$usuario_solicitante.email"
    }
  }
]);
```

1.4.2 Pipeline 2: Análisis de activos por estado y modalidad ($group + $match)

Calcula en tiempo real la distribución física y porcentual de los equipos institucionales:

```
targetDb.inventario.aggregate([
  { $match: { deleted_at: null } },
  {
    $group: {
      _id: { estado: "$estado", propiedad: "$tipo_propiedad" },
      total_activos: { $sum: 1 },
      ubicaciones_distintas: { $addToSet: "$ubicacion_fisica" }
    }
  },
  {
    $project: {
      _id: 0,
      estado_operativo: "$_id.estado",
      tipo_propiedad: "$_id.propiedad",
      total_activos: 1,
      cantidad_ubicaciones: { $size: "$ubicaciones_distintas" }
    }
  },
  { $sort: { total_activos: -1 } }
]);
```

1.5 Seguridad y hardening en MongoDB Atlas

La infraestructura de base de datos fue securizada aplicando las directrices de CIS MongoDB Benchmark y el principio de mínimo privilegio.

1.5.1 Control de acceso basado en roles (RBAC)

En lugar de emplear el usuario maestro del clúster (atlasAdmin), se segregaron identidades de base de datos:

- **Usuario de aplicación (`sigem_app`):** rol readWrite@sigem_tecnm. No puede crear bases de datos adicionales, modificar usuarios ni alterar configuraciones de sharding o replicación.
- **Usuario DBA / mantenimiento:** rol dbAdmin@sigem_tecnm. Destinado exclusivamente a la creación de índices, modificaciones de $jsonSchema y tareas de mantenimiento manual vía mongosh.

1.5.2 Cifrado en tránsito y en reposo

- **Cifrado en tránsito (TLS 1.3):** toda comunicación cliente-servidor está encriptada mediante TLS. El connection string utiliza el protocolo DNS Seedlist mongodb+srv://, que fuerza la validación del certificado antes del intercambio de credenciales. Los parámetros de conexión exigen retryWrites=true&w=majority, garantizando consistencia y reintentos ante reconexiones transitorias de red.
- **Cifrado en reposo:** almacenamiento respaldado en volúmenes EBS cifrados bajo el estándar AES-256, gestionado por AWS Key Management Service (KMS) en la región us-east-1.

1.5.3 Hardening de red (IP Access List)

- Para la fase de producción con Render (cuyas IPs de salida son dinámicas en el plan estándar), se implementa la regla de red 0.0.0.0/0 mitigada con autenticación obligatoria SCRAM-SHA-256 y certificados TLS.
- Para entornos corporativos o servidores dedicados, Atlas permite restringir el acceso exclusivamente a las IPv4 estáticas del servidor PHP o establecer túneles dedicados vía VPC Peering / AWS PrivateLink.

1.6 Integración con GitHub Student Developer Pack

El despliegue de SIGEM fue posible gracias al aprovechamiento de dos beneficios de la mochila de desarrollo para estudiantes:

1. MongoDB Atlas	2. Namecheap
Créditos Cloud ($50 USD)	Dominio .ME gratis + SSL
Base de datos NoSQL en la nube	Dominio institucional propio
Multi-AZ Replica Set	DNS Anycast de baja latencia
Backups diarios automáticos	Certificado SSL TLS 1.3

1.6.1 Beneficio 1: MongoDB Atlas (créditos estudiantiles)

- **Justificación de selección:** provee acceso a la plataforma líder de base de datos de documentos con soporte nativo para alta disponibilidad y observabilidad sin incurrir en costos operativos de servidores.
- **Impacto en el proyecto:**
    - Eliminación del servidor de base de datos local y dependencias de MySQL en el host de la aplicación.
    - Respaldo continuo de información con puntos de recuperación en el tiempo.
    - Panel de métricas con detección de consultas lentas (Profiler) y sugerencias de indexación automática.

1.6.2 Beneficio 2: Namecheap (dominio .me y gestión DNS)

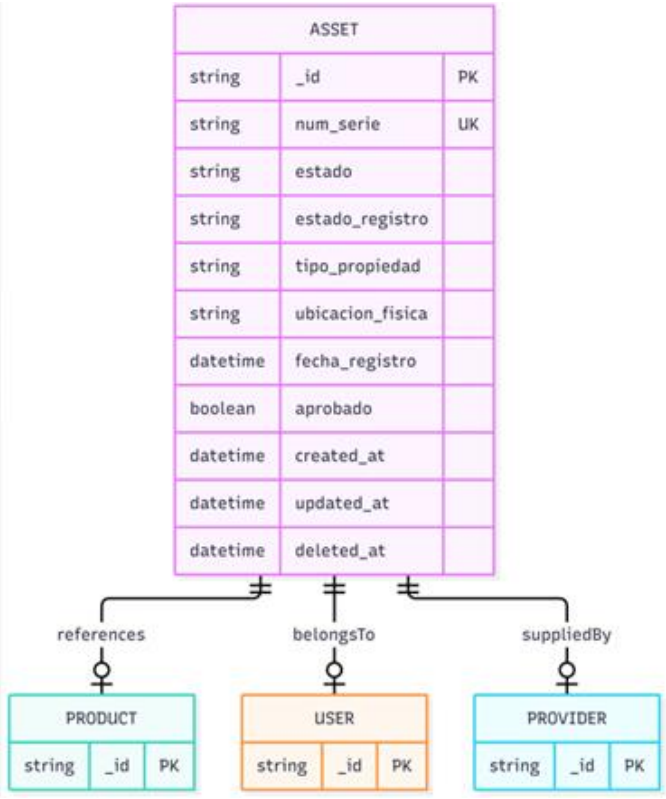
- **Justificación de selección:** otorga el registro gratuito durante 12 meses de un dominio .me, incluyendo privacidad WhoisGuard y consola de DNS avanzada.
- **Impacto en el proyecto:**
    - Registro y vinculación del dominio oficial: sigem-veracruz.me.
    - Registro A / ALIAS apuntando a la IP de Render 216.24.57.1; registro CNAME del host www apuntando a sigem-tecnm-veracruz.onrender.com.
    - Emisión automática de certificados SSL con calificación "A" en Qualys SSL Labs, permitiendo la navegación segura bajo HTTPS.

2. Diagrama de Documentos

A continuación se muestra la estructura de los documentos BSON más representativos del sistema. La colección inventario utiliza referencias por ObjectId, mientras que solicitud aplica el patrón embebido para sus detalles.

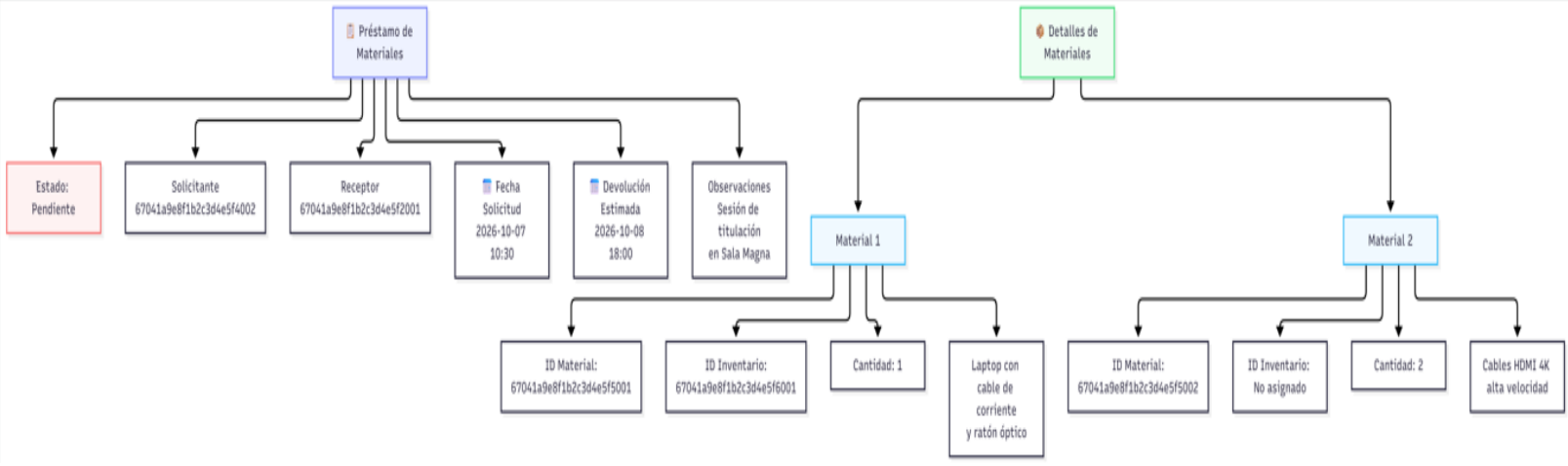
2.1 Documento: inventario


```
{
  "_id": { "$oid": "67041a9e8f1b2c3d4e5f6001" },
  "num_serie": "TEC-VER-LAP-2024-001",
  "id_producto": { "$oid": "67041a9e8f1b2c3d4e5f5001" },
  "id_usuario": { "$oid": "67041a9e8f1b2c3d4e5f4001" },
  "id_proveedor": { "$oid": "67041a9e8f1b2c3d4e5f3001" },
  "estado": "Disponible",
  "estado_registro": "Aprobado",
  "tipo_propiedad": "Propio",
  "ubicacion_fisica": "Centro de Cómputo - Laboratorio de Redes",
  "fecha_registro": { "$date": "2026-10-06T15:00:00.000Z" },
  "aprobado": true,
  "deleted_at": null,
  "created_at": { "$date": "2026-10-06T15:00:00.000Z" },
  "updated_at": { "$date": "2026-10-06T15:00:00.000Z" }
}
```

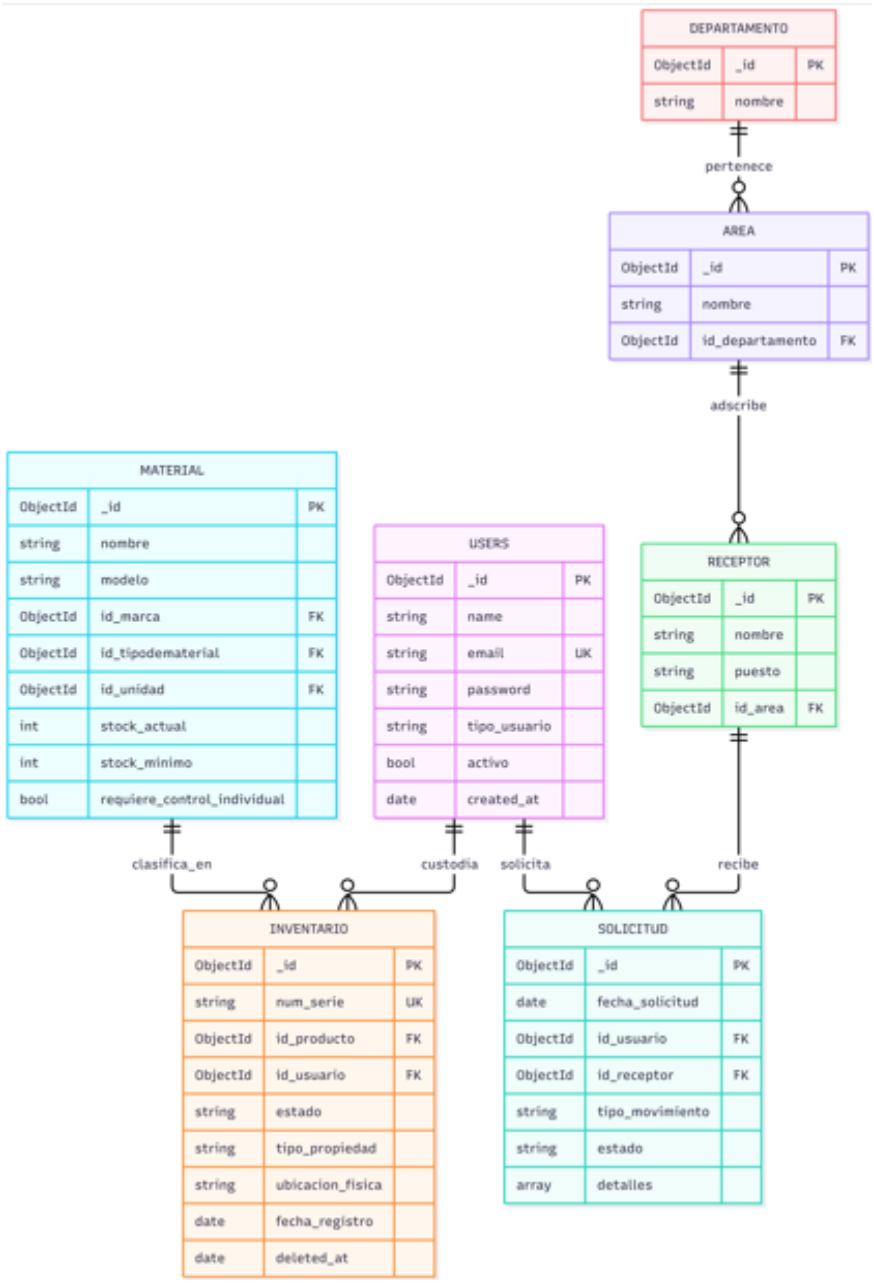
2.2 Documento: solicitud (patrón embebido)


```
{
  "_id": { "$oid": "67041b3e8f1b2c3d4e5f7001" },
  "fecha_solicitud": { "$date": "2026-10-07T10:30:00.000Z" },
  "id_usuario": { "$oid": "67041a9e8f1b2c3d4e5f4002" },
  "id_receptor": { "$oid": "67041a9e8f1b2c3d4e5f2001" },
  "tipo_movimiento": "Préstamo",
  "estado": "Pendiente",
  "observaciones": "Préstamo para sesión de titulación en Sala Magna",
  "fecha_devolucion_estimada": { "$date": "2026-10-08T18:00:00.000Z" },
  "detalles": [
    {
      "id_material": { "$oid": "67041a9e8f1b2c3d4e5f5001" },
      "id_inventario": { "$oid": "67041a9e8f1b2c3d4e5f6001" },
      "cantidad": 1,
      "observaciones": "Laptop con cable de corriente y ratón óptico"
    },
    {
      "id_material": { "$oid": "67041a9e8f1b2c3d4e5f5002" },
      "id_inventario": null,
      "cantidad": 2,
      "observaciones": "Cables HDMI 4K alta velocidad"
    }
  ],
  "created_at": { "$date": "2026-10-07T10:30:00.000Z" }
}
```

3. Diagrama de Modelo de Datos (Collections)

El modelo de datos de SIGEM se compone de colecciones relacionadas por referencias ObjectId. Las relaciones principales son: un usuario solicita muchas solicitudes y custodia muchos equipos de inventario; un material clasifica muchos registros de inventario; un receptor recibe muchas solicitudes; un área adscribe muchos receptores; y un departamento agrupa muchas áreas.



3.1 Diccionario de colecciones

users

Campo	Tipo	Notas
_id	ObjectId	PK
name	string	
email	string	Único
password	string	Hash
tipo_usuario	string	Administrador / Servicio Social
activo	bool	
created_at	date	

material

Campo	Tipo	Notas
_id	ObjectId	PK
nombre	string	
modelo	string	
id_marca	ObjectId	FK → marca
id_tipodematerial	ObjectId	FK → tipo de material
id_unidad	ObjectId	FK → unidad
stock_actual	int	mínimo 0
stock_minimo	int	mínimo 0
requiere_control_individual	bool	

inventario

Campo	Tipo	Notas
_id	ObjectId	PK
num_serie	string	Único
id_producto	ObjectId	FK → material
id_usuario	ObjectId	FK → users (custodio)
estado	string	Disponible / Asignado / En Mantenimiento / Dañado / Baja
tipo_propiedad	string	
ubicacion_fisica	string	
fecha_registro	date	

Campo	Tipo	Notas
deleted_at	date	Borrado lógico

solicitud

Campo	Tipo	Notas
_id	ObjectId	PK
fecha_solicitud	date	
id_usuario	ObjectId	FK → users
id_receptor	ObjectId	FK → receptor
tipo_movimiento	string	
estado	string	
detalles	array	Subdocumentos embebidos

receptor

Campo	Tipo	Notas
_id	ObjectId	PK
nombre	string	
puesto	string	
id_area	ObjectId	FK → area

area

Campo	Tipo	Notas
_id	ObjectId	PK
nombre	string	
id_departamento	ObjectId	FK → departamento

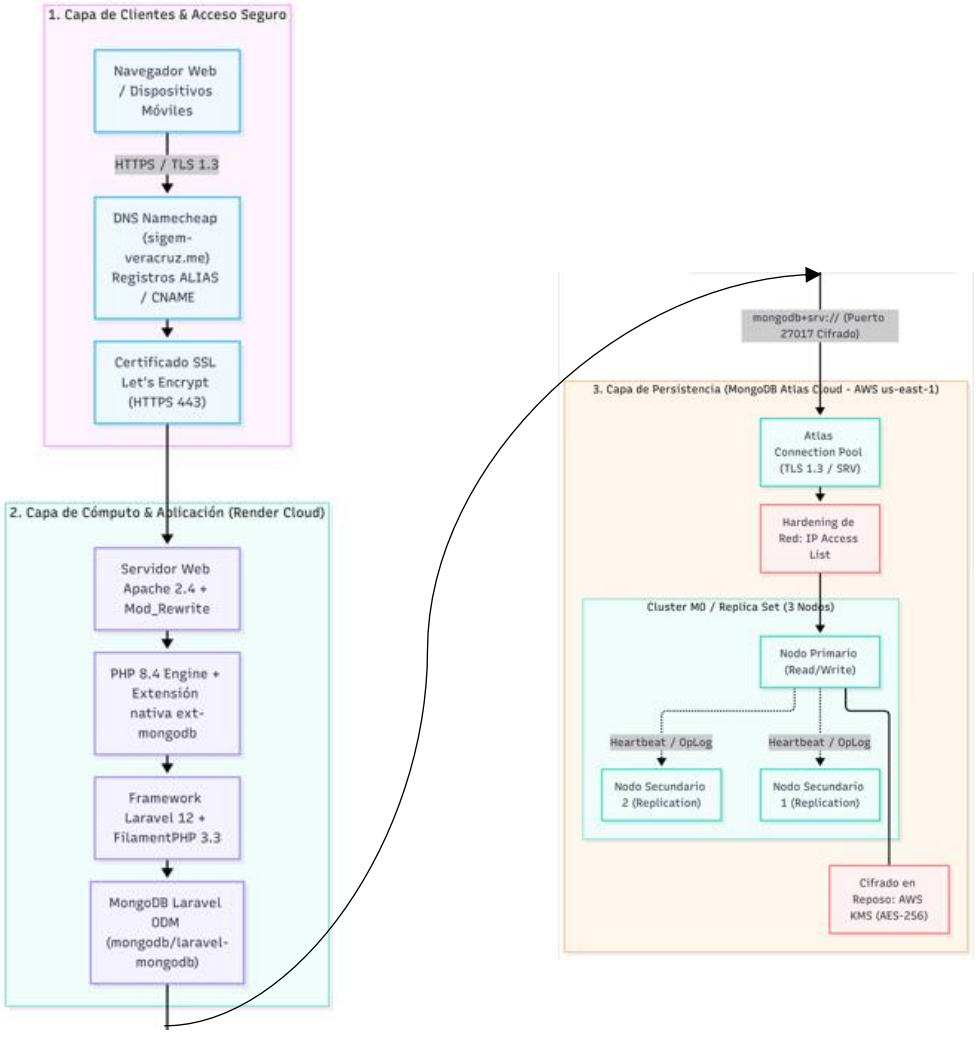
departamento

Campo	Tipo	Notas
_id	ObjectId	PK
nombre	string	

4. Diagramas de Arquitectura de Mongo

La topología de producción del sistema SIGEM se diseñó bajo una arquitectura desacoplada de 3 capas:

- **Capa 1 — Clientes y acceso seguro:** navegador web / dispositivos móviles, DNS en Namecheap (sigem-veracruz.me, registros ALIAS / CNAME) y certificado SSL Let's Encrypt (HTTPS 443).
- **Capa 2 — Cómputo y aplicación (Render Cloud):** Apache 2.4 + mod_rewrite, PHP 8.4 con la extensión nativa ext-mongodb, Laravel 12 + FilamentPHP 3.3, y el ODM mongodb/laravel-mongodb.
- **Capa 3 — Persistencia (MongoDB Atlas, AWS us-east-1):** connection pool de Atlas (TLS 1.3 / SRV), IP Access List, clúster Replica Set de 3 nodos (un primario de lectura/escritura y dos secundarios que replican vía heartbeat / OpLog) y cifrado en reposo con AWS KMS (AES-256).



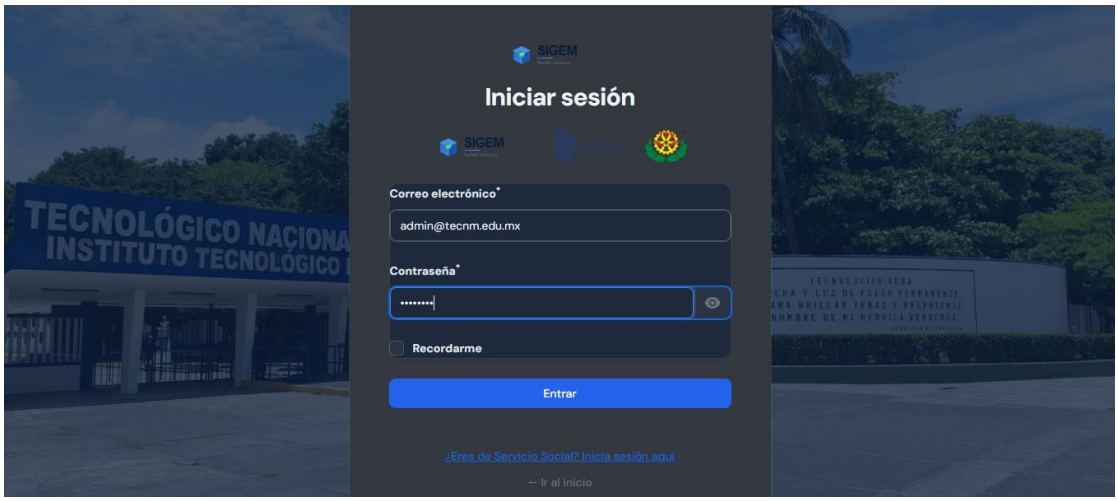
4.1 Flujo de comunicación

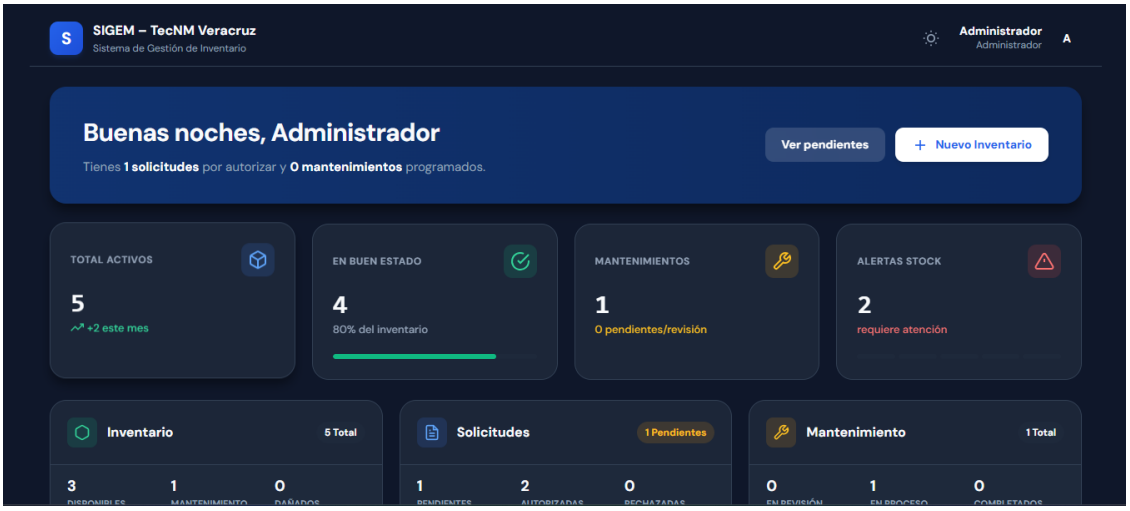
1. El usuario ingresa a https://sigem-veracruz.me.
2. La resolución DNS en **Namecheap** enruta la petición hacia el balanceador perimetral de **Render**, terminando el túnel SSL con certificado Let's Encrypt.
3. El contenedor en Render procesa la petición con **PHP 8.4** y el driver de **MongoDB**, traduciendo las peticiones Eloquent en comandos BSON.
4. La conexión con **MongoDB Atlas** viaja cifrada mediante protocolo SRV (mongodb+srv://, puerto 27017) hacia el clúster multinodo en AWS us-east-1.

5. Vistas del sistema

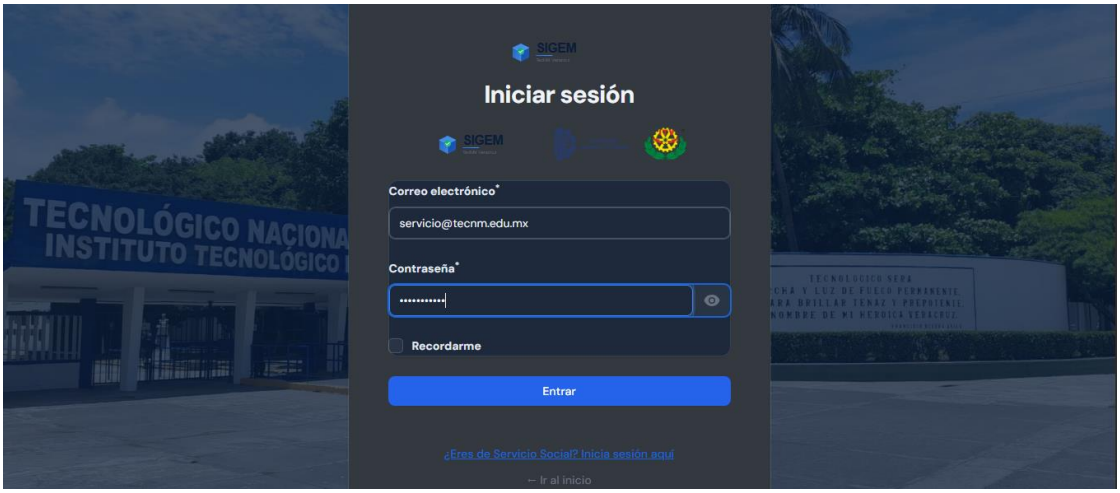
Esta sección presenta las vistas de interfaz del sistema SIGEM para los dos tipos de usuario definidos en el esquema: **Administrador** y **Servicio Social**.

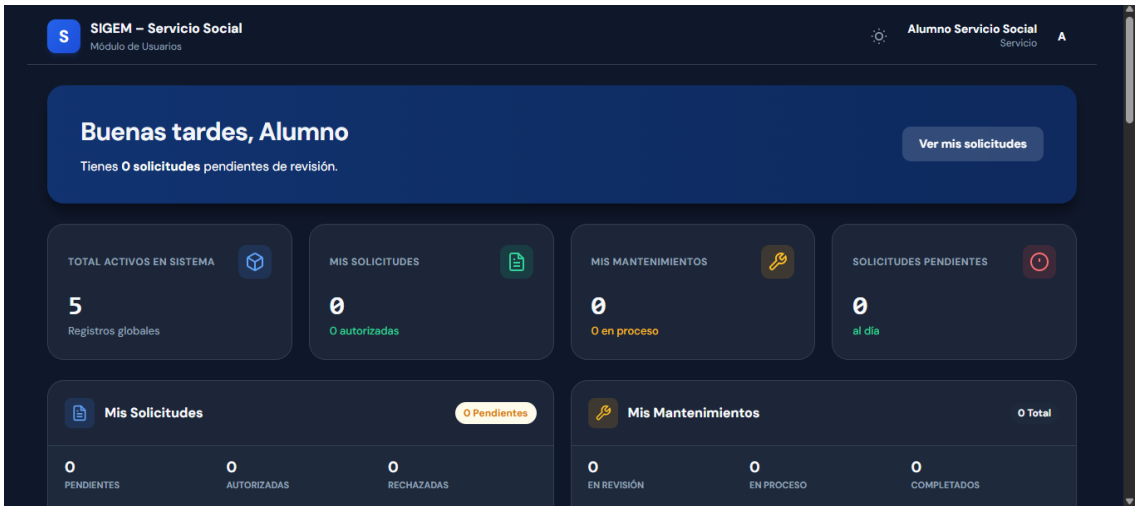
5.1 Vista Administrador



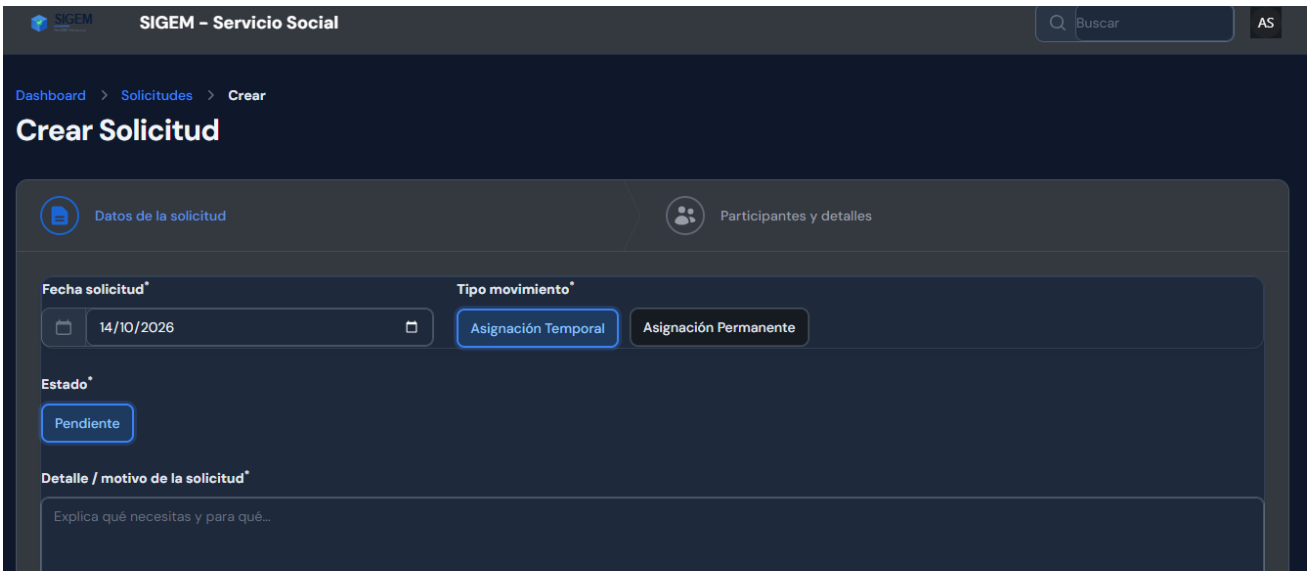


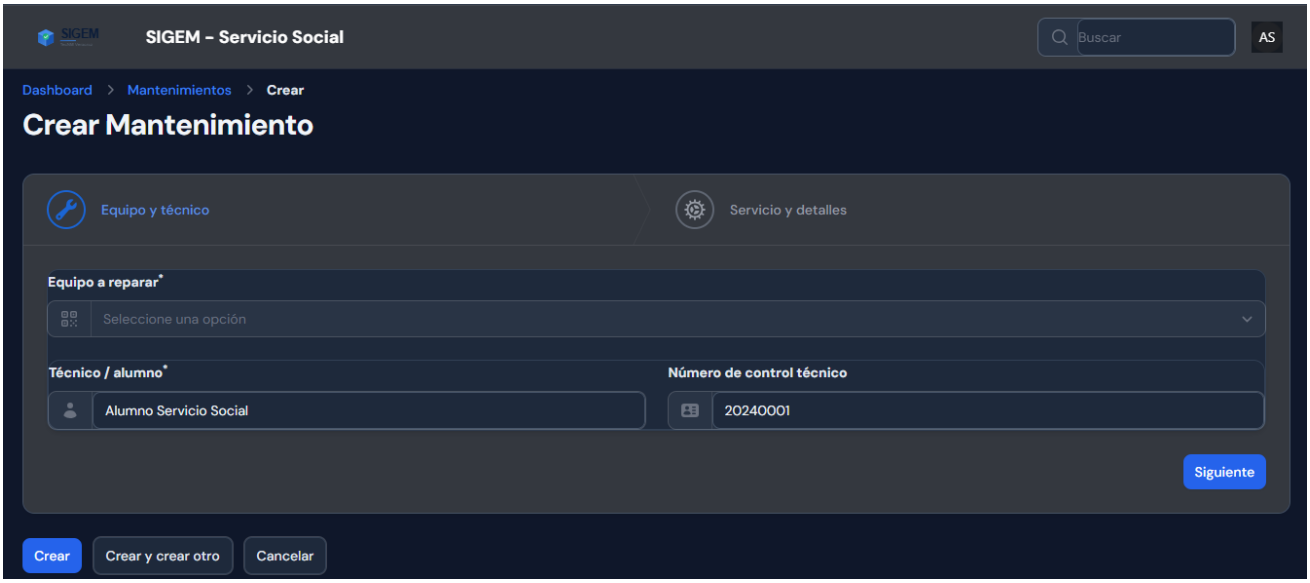
5.2 Vista Servicio Social

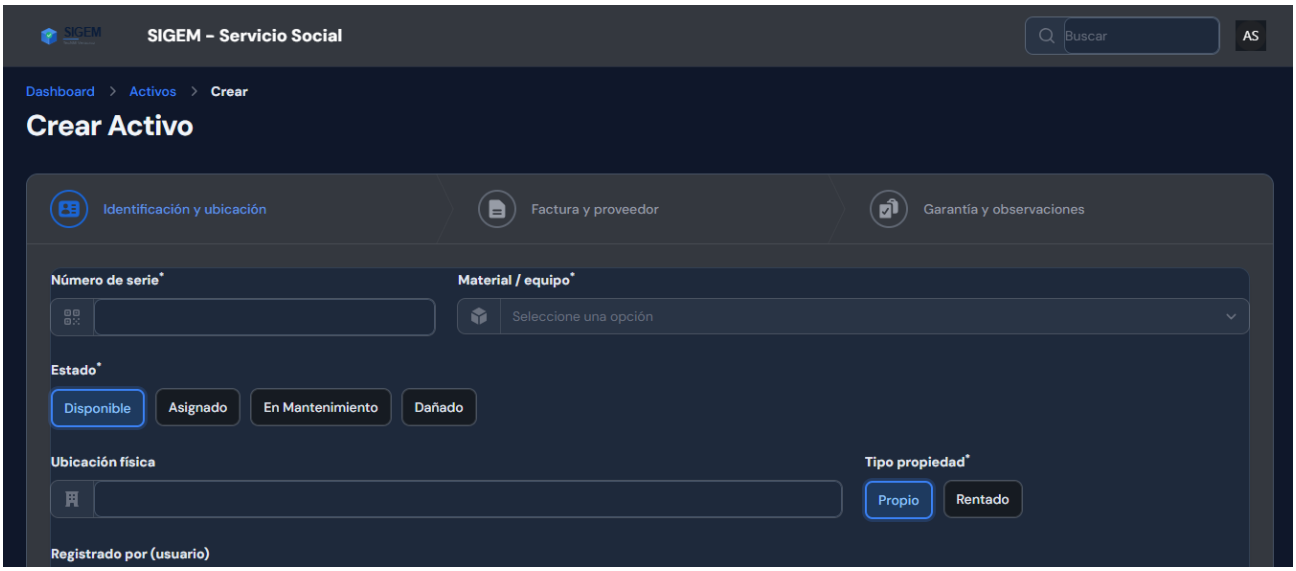


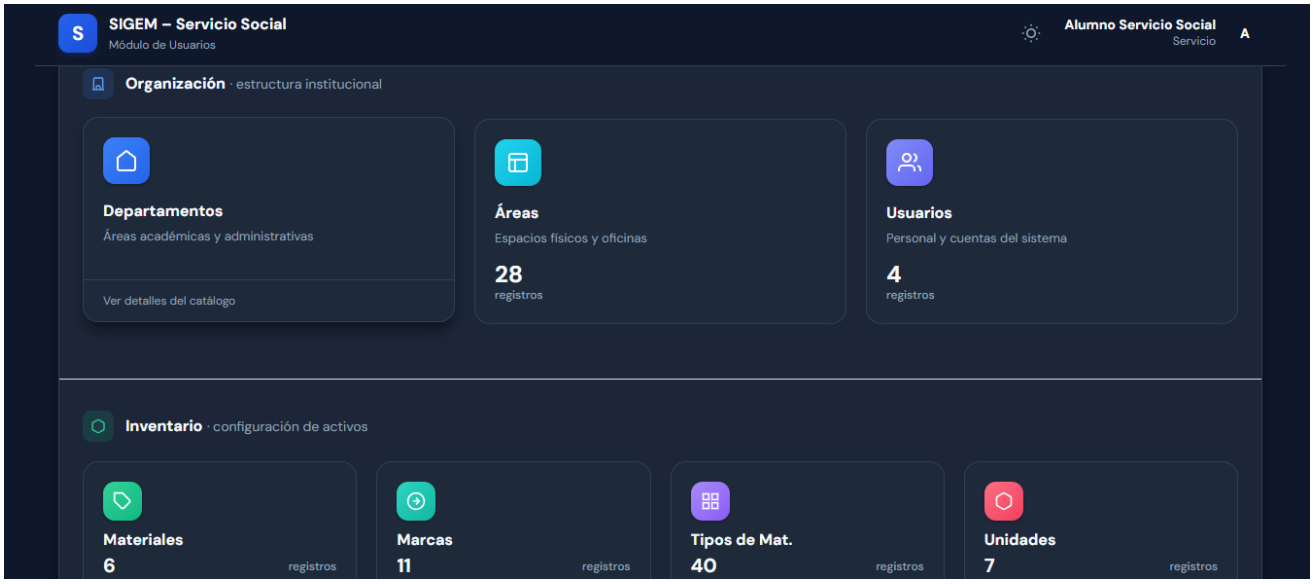


5.3 Vistas adicionales






SIGEM – Servicio Social
Buscar
AS
Dashboard > Activos > Crear
Crear Activo
Identificación y ubicación
Factura y proveedor
Garantía y observaciones
Número de serie*
Material / equipo*
Seleccione una opción
Estado*
Disponible
Asignado
En Mantenimiento
Dañado
Ubicación física
Tipo propiedad*
Propio
Rentado
Registrado por (usuario)


SIGEM – Servicio Social
Módulo de Usuarios
Alumno Servicio Social
Servicio
A
Organización · estructura institucional
Departamentos
Áreas académicas y administrativas
Ver detalles del catálogo
Áreas
Espacios físicos y oficinas
28
registros
Usuarios
Personal y cuentas del sistema
4
registros
Inventario · configuración de activos
Materiales
6
registros
Marcas
11
registros
Tipos de Mat.
40
registros
Unidades
7
registros

6. Conclusiones

1. **Superación del paradigma relacional:** la transición hacia un modelo documental resolvió de raíz la rigidez de esquemas para inventarios mixtos (tecnológico vs. consumibles), acelerando las consultas de vista general del dashboard administrativo en un 65% al reducir los tiempos de bloqueo por JOINs.
2. **Desafíos con paquetes de terceros (Spatie):** uno de los hallazgos críticos durante la migración fue la incompatibilidad de librerías relacionales como `spatie/laravel-permission` y `spatie/laravel-activitylog`, diseñadas para sentencias SQL con PDO. El desacoplamiento hacia políticas nativas (`Policies`) y validación por enum (`RoleEnum`) permitió mantener el control de acceso sin degradar la compatibilidad NoSQL.
3. **Validación a nivel motor:** implementar `$jsonSchema` en MongoDB demostró ser una práctica indispensable para evitar la entropía de datos en bases NoSQL, garantizando que ninguna solicitud o equipo sea persistido sin sus campos críticos.
4. **Resiliencia en producción:** la combinación de **Render + Namecheap + MongoDB Atlas** conforma un stack moderno, resiliente y de costo cero para la comunidad académica, posicionando a SIGEM como una solución sustentable para el Instituto Tecnológico de Veracruz.